Date/Time Functions

- 2Ah - Get system date
- 2Bh - Set system date *
- 2Ch - Get system time
- 2Dh - Set system time *

INT 21h Function 2Ah:
Get system date

- Returns year in CX, month in DH, day in DL, and day of week in AL

```
mov   ah,2Ah
int   21h
mov   year,cx
mov   month,dh
mov   day,dl
mov   dayOfWeek,al
```

INT 21h Function 2Bh:
Set system date

- Sets the system date. AL = 0 if the function was not successful in modifying the date.

```
mov   ah,2Bh
mov   cx,year
mov   dh,month
mov   dl,day
int   21h
cmp   al,0
jne   failed
```

INT 21h Function 2Ch:
Get system time

- Returns hours (0-23) in CH, minutes (0-59) in CL, and seconds (0-59) in DH.

```
mov   ah,2Ch
int   21h
mov   hours,ch
mov   minutes,cl
mov   seconds,dh
```

INT 21h Function 2Dh:
Set system time

- Sets the system date. AL = 0 if the function was not successful in modifying the time.

```
mov   ah,2Dh
mov   ch,hours
mov   cl,minutes
mov   dh,seconds
int   21h
cmp   al,0
jne   failed
```

What's Next

- MS-DOS and the IBM-PC
- MS-DOS Function Calls (INT 21h)
- **Standard MS-DOS File I/O Services**

Standard MS-DOS File I/O Services

- 716Ch - Create or open file
- 3Eh - Close file handle
- 42h - Move file pointer
- 5706h - Get file creation date and time
- Selected Irvine16 Library Procedures

INT 21h Function 716Ch:
Create or open file

- AX = 716Ch
- BX = access mode (0 = read, 1 = write, 2 = read/write)
- CX = attributes (0 = normal, 1 = read only, 2 = hidden, 3 = system, 8 = volume ID, 20h = archive)
- DX = action (1 = open, 2 = truncate, 10h = create)
- DS:SI = segment/offset of filename
- DI = alias hint (optional)

Example: Create a New File

```
mov   ax,716Ch                ; extended open/create
mov   bx,2                    ; read-write
mov   cx,0                    ; normal attribute
mov   dx,10h + 02h            ; action: create + truncate
mov   si,OFFSET Filename
int   21h
jc    failed
mov   handle,ax               ; file handle
mov   actionTaken,cx          ; action taken to open file
```

Example: Open an Existing File

```
mov   ax,716Ch                  ; extended open/create
mov   bx,0                      ; read-only
mov   cx,0                      ; normal attribute
mov   dx,1                      ; open existing file
mov   si,OFFSET Filename
int   21h
jc    failed
mov   handle,ax                 ; file handle
mov   actionTaken,cx            ; action taken to open file
```

Selected Irvine16 Library Procedures

- 16-Bit ReadString procedure

ReadString Procedure

The ReadString procedure from the Irvine16 library reads a string from standard input and returns a null-terminated string. When calling it, pass a pointer to a buffer in DX. Pass a count of the maximum number of characters to input, plus 1, in CX. Writestring inputs the string from the user, returning when either of the following events occurs:

1. CX –1 characters were entered.
2. The user pressed the Enter key.

```
.data
buffer BYTE 20 DUP(?)
.code
mov  dx,OFFSET buffer
mov  cx,LENGTHOF buffer
call ReadString
```

ReadString Procedure

You can also call it using 32-bit registers:

```
.data
buffer BYTE 20 DUP(?)
.code
mov  edx,OFFSET buffer
mov  ecx,LENGTHOF buffer
call ReadString
```

ReadString returns a count of the number of characters actually read in the EAX register.

ReadString Implementation

```
ReadString PROC
    push cx                 ; save registers
    push si
    push cx                 ; save character count
    mov  si,dx              ; point to input buffer
    dec  cx                 ; save room for null byte
L1: mov  ah,1               ; function: keyboard input
    int  21h                ; returns character in AL
    cmp  al,0Dh             ; end of line?
    je   L2                 ; yes: exit
    mov  [si],al            ; no: store the character
    inc  si                 ; increment buffer pointer
    loop L1                 ; loop until CX=0
L2: mov  BYTE PTR [si],0    ; insert null byte
    pop  ax                 ; original digit count
    sub  ax,cx              ; AX = size of input string
    pop  si                 ; restore registers
    pop  cx
    ret
ReadString ENDP             ; returns AX = size of string
```